

学号： 1906410229

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 随机化算法

2022年秋季学期

上机时间：第 11 周 星期 五 第 3 ~ 4 节

学生姓名： 蔡艺洵 专业班级： 计算机 1902

指导教师： 刘俊岭 成 绩：

项 目 名 称	随机化法	学 时	4
日期： 2022 年 11 月 11 日		第 11 教学周	
硬件及软件			
名 称		版本、型号	
Dev-C++		5.11	
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度与空间复杂度。 本次任务： 实验 7-1：计算定积分(随机化算法) 实验 7-2：舍伍德快速排序(随机化算法) 实验 7-3：拉斯维加斯算法实现 nQueen(随机化算法)			

实验 7-1 计算定积分(随机化算法)

1.问题描述:

用随机投点法计算定积分。

2.算法实例:

输入: $f(x)=x*x$, 区间 $a = 0.0, b = 1.0$ 投点次数 $n=100$

输出: 0.330632

要求:

- 1 填空完成程序;
 - 2 输出不同投点次数时定积分的值。
 - *3. 改变函数表达式、区间值为 $f(x)=1/3*x*x*x$, 区间 $a = 1.0, b = 3.0$, 计算定积分。
- 输出:

G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-1.exe

```
用随机投点法计算定积分
f(x)=x*x, 范围[0, 1], 真值为1/3
随机投掷100个点时, 定积分=0. 328433
随机投掷1000个点时, 定积分=0. 336483
随机投掷10000个点时, 定积分=0. 336655
随机投掷100000个点时, 定积分=0. 331556
随机投掷1000000个点时, 定积分=0. 333916
随机投掷10000000个点时, 定积分=0. 333404
请按任意键继续. . .
```

输入的表达式为 $f(x)=1/3*x*x*x$, 邻区间 $a = 1.0, b = 3.0$, 计算定积分:

G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-1.exe

```
用随机投点法计算定积分
f(x)=1/3*x*x*x, 范围[1, 3], 真值为
随机投掷100个点时, 定积分=7. 096964
随机投掷1000个点时, 定积分=6. 848610
随机投掷10000个点时, 定积分=6. 703619
随机投掷100000个点时, 定积分=6. 689709
随机投掷1000000个点时, 定积分=6. 663835
随机投掷10000000个点时, 定积分=6. 664975
请按任意键继续. . .
```

3.程序代码:

```
/**
随机化算法 用随机投点法计算定积分
*/

#define _CRT_SECURE_NO_WARNINGS

#include "iostream"
#include "ctime"
#include "iomanip"
```

```

using namespace std;

const unsigned long maxshort = 65536L;
const unsigned long multiplier = 1194211693L;
const unsigned long adder = 12345L;

class RandomNumber
{
private:
    unsigned long randSeed; //当前种子
public:
    RandomNumber(unsigned long s = 0); //构造函数，默认值 0 表示由系统自动产生种子
    unsigned short Random(unsigned long n); //产生 0:n-1 之间的随机整数
    double fRandom(void); //产生[0:1)之间的随机实数
};

RandomNumber::RandomNumber(unsigned long s) //产生种子
{
    if (s == 0)
        randSeed = time(0); //用系统时间产生种子
    else
        randSeed = s; //用户提供种子
}

unsigned short RandomNumber::Random(unsigned long n) //产生 0:n-1 之间的随机整数
{
    randSeed = multiplier * randSeed + adder;
    return (unsigned short)((randSeed >> 16) % n); //高 16 位随机性较好，右移 16 位后，映射到 (0~n-1)
    范围内
}

double RandomNumber::fRandom(void) //产生[0,1)之间的随机整数
{
    return Random(maxshort) / double(maxshort); //产生 0~(maxshort-1)间的随机整数，在除以 maxshort
}

double Darts(int n, double a, double b);
double f(double x);

int main()
{
    double a = 0.0, b = 1.0;
    double integral;
    long num = 100;
    printf("用随机投点法计算定积分\n");
    printf("f(x)=1/3*x*x*x,范围[1,3],真值为\n");

```

```

    for (int i = 1; i < 7; i++){
        integral = Darts(num,a,b);
        printf("随机投掷%d 个点时， ", num);
        printf("定积分=%f\n", integral);
        num = num * 10;
    }

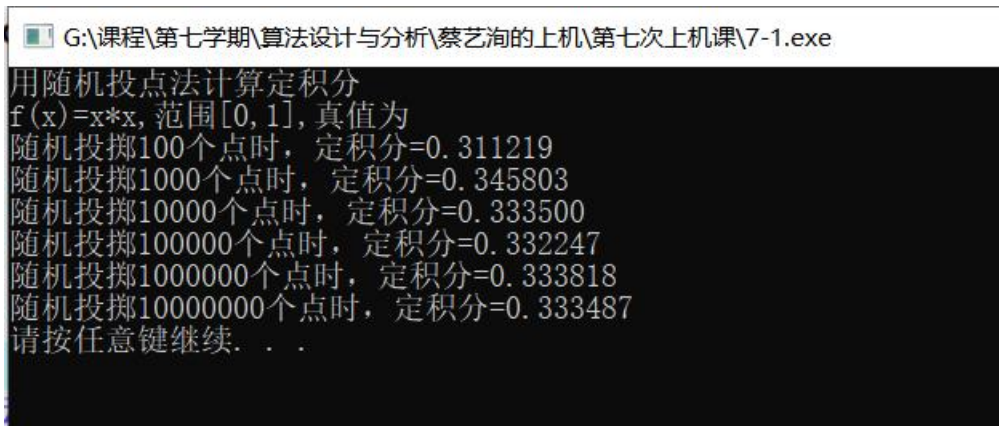
    system("PAUSE");
    return 0;
}
/*
* 基本思想是在矩形区域内随机均匀投点，求出由这些点
* 产生的函数值的算术平均值，再乘以区间宽度，即可得
* 出定积分的近似解
*/
double Darts(int n, double a, double b)
{
    static RandomNumber dart;
    double sum = 0.0;
    for (int i = 0; i < n; i++)
    {
        double x = (b - a)*dart.fRandom()+ a;//产生[a,b)之间的随机数
        sum = sum + f(x);
    }
    return (b - a)*sum / n;
}

double f(double x)
{
    return x*x;
    //return ((double)1/3)*x*x*x;
}

```

4.程序验证:

测试 1: 输入的例子为: $f(x)=x^2$, 区间 $a = 0.0$, $b = 1.0$ 投点次数 $n=100$ 、 1000 、 10000 、 100000 、 1000000 时, 其输出的结果是否接近 0.33, 如果接近, 则算法正确。



```

G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-1.exe
用随机投点法计算定积分
f(x)=x*x, 范围[0, 1], 真值为
随机投掷100个点时, 定积分=0. 311219
随机投掷1000个点时, 定积分=0. 345803
随机投掷10000个点时, 定积分=0. 333500
随机投掷100000个点时, 定积分=0. 332247
随机投掷1000000个点时, 定积分=0. 333818
随机投掷10000000个点时, 定积分=0. 333487
请按任意键继续. . .

```

测试 2: 输入的例子为: $f(x)=1/3*x*x*x$, 区间 $a=1.0, b=3.0$ 投点次数 $n=100, 1000, 10000, 100000, 1000000$ 时, 按照测试 1 的样式进行结果输出。

G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-1.exe

```
用随机投点法计算定积分
f(x)=1/3*x*x*x, 范围[1, 3], 真值为
随机投掷100个点时, 定积分=6.403766
随机投掷1000个点时, 定积分=6.828662
随机投掷10000个点时, 定积分=6.534207
随机投掷100000个点时, 定积分=6.678775
随机投掷1000000个点时, 定积分=6.674894
随机投掷10000000个点时, 定积分=6.665704
请按任意键继续. . .
```

5. 算法分析:

该算法使用了数值随机算法。利用随机数发生器的算法, 算法执行的某些选择依赖于随机数发生器所产生的随机数。其时间复杂度为 $O(n)$, n 为投点尝试数。使用了长度为 1 的记录变量, 所以空间复杂度为 1,

时间复杂度: $O(n)$

空间复杂度: $O(1)$

实验 7-2 舍伍德快速排序(随机化算法)

1. 问题描述:

利用舍伍德算法实现快速排序。

2. 算法实例:

输入 100 数值。

要求:

1 填空完成程序;

2 输出排序结果。

*3 设计一个划分算法, 利用 3 个随机数的中位数作为 pivot。

输出的结果有两种, 第一种是前面两个问题的算法, 只随机产生一个数, 把这个数作为 pivot 进行的排序, 第二种输出为针对第三问, 产生三个随机数, 把三个随机数的中位数作为 pivot 进行排序, 这种输出, 我们要知道三个随机数是哪三个, 我们是否正确选择了中位数。两种输出如下图所示。

```
G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-2.exe
原次序:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43
44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83
84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
第一次洗牌:
57 4 48 31 41 3 88 76 91 96 13 68 32 58 24 42 27 63 47 25 56 34 2 70 19 23 74 9 87 64 12 86 60 65 21 1 33 100 20 11 90 4
4 28 71 97 35 7 37 17 18 26 61 81 29 52 67 77 36 55 38 8 6 62 73 92 93 43 16 89 46 85 49 80 39 45 78 75 82 83 50 99 10 1
5 66 54 79 22 53 72 14 51 30 40 69 5 98 84 95 94 59
第二次洗牌:
77 31 37 12 90 48 53 78 51 98 32 16 86 36 70 44 74 62 7 19 67 65 4 46 47 2 39 91 22 73 68 79 38 92 56 57 60 59 25 13 14
71 9 85 84 21 88 33 27 63 23 8 99 87 61 43 75 1 52 100 76 3 6 80 30 40 28 42 72 35 54 17 50 20 97 82 45 10 15 18 94 96 2
4 93 29 83 34 81 49 58 26 64 11 89 41 95 66 5 69 55
选择的pivot位置, 值序列为:
58:52,
20:18,
3:12,
10:2,
7:11,
4:5,
2:3,
8:8,
5:6,
8:10,
15:15,
12:13,
13:16,
50:23,
20:21,
18:20,
33:32,
```

第一种输出, 只随机产生一个数, 即为 pivot。

```
选择 G:\课程\第七学期\算法设计与分析\蔡艺淘的上机\第七次上机课\7-2.exe
原次序:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43
44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83
84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
第一次洗牌:
100 93 70 50 33 99 36 90 79 60 62 1 85 26 54 65 76 58 22 18 83 30 96 71 84 97 89 57 98 94 87 37 28 86 38 51 32 44 16 34
31 39 91 77 42 40 5 61 81 64 41 13 9 78 24 56 69 23 68 47 74 46 63 14 92 80 27 95 35 75 29 59 48 72 11 20 6 12 7 19 73 5
2 4 15 25 88 3 45 66 67 10 17 2 49 8 53 43 82 55 21
第二次洗牌:
21 2 75 64 28 55 51 67 7 47 46 100 25 97 78 92 20 23 30 58 4 94 53 29 15 43 66 69 82 49 3 32 57 88 44 41 37 77 65 86 87
16 10 6 39 34 33 74 73 14 31 85 79 12 71 56 35 96 95 5 72 40 63 26 17 19 89 8 38 11 98 68 61 59 62 18 99 1 36 22 48 13 5
0 54 84 45 70 42 80 27 60 76 93 81 90 9 91 52 24 83
选择的pivot位置, 值序列为:
k1=20 k2=70 k3=57
k=57
57:96,
k1=45 k2=30 k3=12
k=30
30:3,
k1=0 k2=0 k3=1
k=0
0:1,
k1=51 k2=57 k3=92
k=57
57:21,
k1=11 k2=15 k3=5
k=11
11:5,
k1=15 k2=5 k3=7
k=7
```

第二种输出, 产生三个随机数, 其中位数为 pivot。

3.程序代码:

/*exp7-4 QuickSortSherwood

舍伍德快速排序。虽然快速排序平均复杂度可以看做 $n\log n$ 。

但算法复杂度与输入实例之间有关系, 未必都是平均时间复杂度。

这是采用舍伍德快速排序的原因, 消除掉算法复杂度与输入实例之间的关系。

*/

#define _CRT_SECURE_NO_WARNINGS

#include<iostream>

#include<stdlib.h>

#include<time.h>

#include<cstdio>

#include<algorithm>

#define MAX 100

using namespace std;

int Partition(int a[], int low, int high)

{

int key = a[low];

int i = low, j = high + 1;

while (1)

{ while (a[++i]<key&&i <= high);

while (a[--j]>key&&j >= low);

if (i >= j) break;

if (a[i] != a[j])

swap(a[i], a[j]);

}

if ((j != low) && (a[low] != a[j]))

swap(a[low], a[j]);

```

        return j;
    }
//随机得到一个位置进行划分操作
int RandPartition(int a[], int low, int high)
{
    srand(time(NULL));
    //随机生成一个 low,high 之间的位置 k
    // int k1 = rand() % (high - low + 1) + low;
    // int k2 = rand() % (high - low + 1) + low;
    // int k3 = rand() % (high - low + 1) + low;
    // cout <<"k1="<<k1 <<" "<<"k2="<<k2<<" "<<"k3="<<k3<<" "<<endl;
    // int k;
    // if((k1>=k2&& k1<=k3)|| (k1>=k3&& k1<=k2)){
    //     k=k1;
    // }
    // else if((k2>=k1&& k2<=k3)|| (k2>=k3&& k2<=k1)){
    //     k=k2;
    // }
    // else{
    //     k=k3;
    // }
    // cout <<"k="<<k<<endl;
    int k = rand() % (high - low + 1) + low;
    cout << k << ":" << a[k] << ", "<<endl;
    //将 a[k]作为 pivot
    swap(a[k],a[low]);
    k = Partition(a, low, high);
    return k;
}
void Quick_sort(int a[], int low, int high)
{
    if (low<high)
    {
        int k = RandPartition(a, low, high);
        Quick_sort(a, low, k-1);
        Quick_sort(a, k+1, high);
    }
}
// 随机洗牌算法
void random_shuffle(int a[]) {

    srand(time(NULL));
    for (int i = MAX - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        int temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}

```



```

    }
}
int main()
{
    int i;
    int a[MAX];
    for (i = 0; i < MAX; i++)
        a[i] = i + 1;
    cout << "原次序: ";
    cout << endl;
    for (i = 0; i < MAX; i++)
        cout << a[i] << " ";
    cout << endl;
    random_shuffle(a); //To randomize the array
    //cout << "排序前: ";
    cout << "第一次洗牌: ";
    cout << endl;
    for (i = 0; i < MAX; i++)
        cout << a[i] << " ";
    cout << endl;
    random_shuffle(a); //To randomize the array
    //cout << "排序前: ";
    cout << "第二次洗牌: ";
    cout << endl;
    for (i = 0; i < MAX; i++)
        cout << a[i] << " ";
    cout << endl;
    cout << "选择的 pivot 位置, 值序列为: ";
    cout << endl;
    Quick_sort(a, 0, MAX-1);
    cout << endl;
    cout << "排序后: ";
    for (i = 0; i < MAX; i++)
        cout << a[i] << " ";
    cout << endl;
    system("PAUSE");
    return 0;
}

```

4.程序验证:

输入的数据为 100 个乱序的数字, 对其进行快速排序。看是否能按照数字的大小进行排序, 第一种输出为产生一个随机数, 即为 pivot, 以此进行快速排序, 运行程序得到以下截图:

```
G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-2.exe
62:67,
60:66,
62:61,
63:62,
62:64,
67:68,
68:70,
85:94,
87:77,
73:73,
73:75,
83:92,
79:84,
77:83,
79:79,
79:80,
80:82,
86:89,
86:88,
84:87,
84:86,
89:91,
94:99,
96:98,
94:96,
排序后: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41
1 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81
1 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
请按任意键继续. . .
```

第二种输出为针对第三问的要求，将产生三个随机数，其中位数作为 pivot,以此进行快速排序，运行结果如下图：

```
G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-2.exe
84:84,
k1=86 k2=86 k3=86
k=86
86:85,
k1=86 k2=86 k3=85
k=86
86:87,
k1=95 k2=97 k3=94
k=95
95:95,
k1=91 k2=91 k3=92
k=91
91:91,
k1=93 k2=93 k3=93
k=93
93:93,
k1=95 k2=97 k3=99
k=97
97:98,
k1=96 k2=96 k3=95
k=96
96:96,
k1=99 k2=99 k3=98
k=99
99:99,
排序后: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41
1 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81
1 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
请按任意键继续. . .
```

程序验证正确，可以得到 100 个数字的快速排序，如上图的结果。

5.算法分析：

本算法求解的问题是快速排序问题，在传统的快速排序算法的基础上，使用舍伍德(Sherwood)算法进行优化，而舍伍德算法随机地选择一个数组元素作为划分基准，引入随机性将它改造成一个舍伍德算法，用来消除或减少问题的不同实例之间在计算时间上的差别。舍伍德算法的精髓不是避免算法的最坏情况行为，而是设法消除这种最坏行为与特定实例之间的关联性。

其时间复杂度为 $O(n\log n)$ 。空间复杂度，本次算法只用了常量级的变量进行存储，所以空间复杂度为 $O(1)$ 。所以

- 时间复杂度： $O(n\log n)$
- 空间复杂度： $O(1)$

实验 7-3 拉斯维加斯算法实现 nQueen(随机化算法)

1.问题描述:

nQueen 随机放置位置策略和回溯法相结合求解 n 皇后问题.

拉斯维加斯算法不会得到不正确的解。一旦用拉斯维加斯算法找到一个解，这个解就一定是正确解。但有时用拉斯维加斯算法找不到解。与蒙特卡罗算法类似，拉斯维加斯算法找到正确解的概率随着它所用的计算时间的增加而提高。对于所求解问题的任一实例，用同一拉斯维加斯算法反复对该实例求解足够多次，可使求解失败的概率任意小。

N 皇后的求解方法:

消极: LV 算法过于消极，一旦失败，从头再来。

乐观: 回溯算法过于乐观，一旦放置某个皇后失败，就进行系统回退一步的策略，而这一步往往不一定有效

折中: 一般情况下更好，先用 LV 方法随机的放置前若干个结点，然后使用回溯法放置后若干个结点

2.算法实例:

示例 1: 皇后数 n=12 随机放置皇后数 stop=5

要求:

- 1 填空完成程序;
- 2 输出放置结果，运行次数。
- *3 改变皇后数与随机放置皇后数，输出结果。

```
Queen X;  
X.n = n;  
int *p = new int[n + 1];  
int *q = new int[n + 1];  
for (int i = 0; i <= n; i++){  
    p[i] = 0;  
    q[i] = 0;  
}  
X.y = p;  
X.x = q;  
// 设置随机放置皇后的个数  
int stop = 5;  
if (n>15)  
    stop = n - 15;  
bool found = false;
```

G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-3.exe

12皇后问题的解为 (随机放置的皇后数为5):
5 9 11 3 7 2 8 1 12 10 6 4
运行的次数为: 1
请按任意键继续. . .

3.程序代码:

/*nQueen 随机放置位置策略和回溯法相结合求解 n 皇后问题.

*/

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <math.h>
```

```
#include <time.h>
```

```
using namespace std;
```

```

//利用线性同余法产生的伪随机数
const unsigned long maxshort = 65536;
const unsigned long multipliter = 1194211693L;
const unsigned adder = 12345L;
class RandomNumber{
private:
    unsigned long randSeed;
public:
    RandomNumber(unsigned long s = 0);
    unsigned short Random(unsigned long n);
    double fRandom(void);
};
RandomNumber::RandomNumber(unsigned long s){
    if (s == 0)
        randSeed = time(0);
    else
        randSeed = s;
}
unsigned short RandomNumber::Random(unsigned long n){
    randSeed = multipliter*randSeed + adder;
    return (unsigned short)((randSeed >> 16) % n);
}
double RandomNumber::fRandom(void){
    return Random(maxshort) / double(maxshort);
}
class Queen{
    friend bool nQueen(int);
private:
    bool Place(int k);
    bool Backtrack(int t);
    bool QueensLV(int stop);
    int n, *x, *y;
};
bool Queen::Place(int k){    //测试皇后 k 置于第 x[k]列的合法性
    for (int j = 1; j<k; j++)
        if ((abs(k - j) == abs(x[j] - x[k])) || (x[j] == x[k]))
            return false;
    return true;
}
bool Queen::Backtrack(int t){
    if (t>n){
        for (int i = 1; i <= n; i++)
            y[i] = x[i];
        return true;
    }
    else{
        for (int i = 1; i <= n; i++){

```

```

        x[t] = i;
        if (Place(t) && Backtrack(t + 1))
            return true;
    }
}
return false;
}

bool Queen::QueensLV(int stop){//随机放置 n 个皇后的拉斯维斯加算法
    RandomNumber rnd;    // 随机数产生器
    int k = 1;            //下一个放置的皇后的编号
    int count = 1;
    while ((k <=stop) && (count>0)){
        count = 0;
        for (int i = 1; i <= n; i++){//遍历第 k 行所有可以放置的列号，用 y 保存，并用 count 记录有多少个
位置可以放置
            x[k] = i;
            if (Place(k))
                y[count++] = i;
        }
        if (count>0)    //利用上面保存的可以放置的列，然后随机取其中一列来放置第 k 行的皇后。是
Las Vegas 思想
            x[k++] = y[rnd.Random(count)];//随机位置
    }
    return (count>0); //count>0 表示放置成功
}

bool nQueen(int n){
    Queen X;
    X.n = n;
    int *p = new int[n + 1];
    int *q = new int[n + 1];
    for (int i = 0; i <= n; i++){
        p[i] = 0;
        q[i] = 0;
    }
    X.y = p;
    X.x = q;
    // 设置随机放置皇后的个数
    int stop = 5;
    if (n>15)
        stop = n - 15;
    bool found = false;
    while (!X.QueensLV(stop));
    // 算法的回溯搜索部分
    if (X.Backtrack(stop + 1)){
        for (int i = 1; i <= n; i++)
            cout << p[i] << " ";
    }
}

```

```

        found = true;
    }
    cout << endl;
    delete[] p;
    delete[] q;
    return found;
}
int main(){
    int n = 12;
    int k=1;

    cout << n << "皇后问题的解为（随机放置的皇后数为 5）： " << endl;
    while (!nQueen(n)){
        k++;
    }
    cout << "运行的次数为： " << k << endl;
    system("PAUSE");
    return 0;
}

```

4.程序验证:

验证例子 1：输入的皇后数为 12，随机放置皇后数为 5，运行的次数以及皇后其中的放置位置，如图所示：

 G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-3.exe

```

3 9 12 10 2 11 6 4 1 8 5 7
运行的次数为： 4791
请按任意键继续. . .

```

当更改皇后数，求解 14 皇后问题，随机放置的皇后数为 7. 结果如图所示：

```

int stop = 7;
if (n>15)
    stop = n - 15;
bool found = false;
while (!X.QueensLV(stop));
// 算法的回溯搜索部分
if (X.Backtrack(stop + 1)){
    for (int i = 1; i <= n; i++)
        cout << p[i] << " ";
    found = true;
}
cout << endl;
delete[] p;
delete[] q;
return found;

int main(){
    int n = 14;

```

 G:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第七次上机课\7-3.exe

```

14皇后问题的解为（随机放置的皇后数为7）：
7 1 6 9 13 8 4 2 10 14 11 5 3 12
运行的次数为： 1
请按任意键继续. . .

```

5.算法分析：

时间复杂度： $O(n^2)$
空间复杂度： $O(n)$

分析与总结：

随机化算法的基本特征就是对所求解问题的同一实例用同一随机化算法求解两次可能得到完全不同的效果。使用随机策略有时候可以对原本的算法进行一定程度上的优化，从而改善算法。

本次学习随机化算法一共有四种，分别是概率数值法、舍伍德算法、拉斯维加斯算法、蒙特卡洛法，前三种我接触毕竟少，第四种我接触过不少，因为我之前搞计算机博弈研究时，五子棋算法的两大主流算法分别是博弈树剪枝和蒙特卡洛算法，所以对蒙特卡洛算法很熟悉。对于四种随机化算法的比较，它们各有特点，例如有时候拉斯维加斯算法找不到解，而另外三种都能找到解，而随机数值法只能求得近似解，而蒙特卡洛算法一定能找到解，但这个解不一定是正确解，舍伍德算法能找到解、而且保证这个解是正确解，但是舍伍德算法不一定能优化原来的算法，它只是用来消除输入数据与最坏情况的关联性。所以四种算法各有特点，需要因地制宜的使用对应的算法来优化。

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：